Execution Plan

What to build first, and how we keep today's modules safe

The promise

Everything already working (login, employees, leave, attendance and biometric, WFH, travel allowance, projects, tasks, Hubstaff, bug tracker, chat, recognitions, issues) keeps working at every step. We get there with **five rules**, a **safety net built before any new feature**, and a **release routine** for every phase.

The five rules

• **Add, never change.** New features go in new tables, new controllers and new pages. We do not rewrite existing ones.

• **Safe database changes only.** New tables, and new columns that are optional (nullable or with a default). No rename, no delete, no changing a column type on live tables. Any clean-up happens months later, after the new flow is proven.

• **Everything behind a switch.** Each new module has an on/off flag (the portal already has a settings table we can reuse) and is visible only to the new roles. Off = the portal looks exactly as today.

• **New roles, old roles untouched.** Sales, Accounts, PMO, PC and the rest are new roles. We never edit what Employee, Team Lead, HR and Super Admin can do today.

• **Nothing reaches live without passing the safety net** (tests + staging check + backup + a known way to roll back).

Where today's setup is risky

Today	What could go wrong	What we do
Live updates by **git pull of main** in ~/public_html/api/Inter-Smart-Portal. An old unused copy sits in ~/public_html/backend, and live runs with debug on (APP_ENV=local)	A push to main can reach live as soon as someone pulls it; a wrong-folder deploy is easy; debug mode exposes internals.	Work only on feature branches. Remove the old copy, set production mode, and write the one deploy path into deploy.sh.
No automatic checks on code (no CI)	Nobody notices a broken module until a user does.	Add GitHub Actions to run tests on every pull request.
Only 13 backend test files, no frontend tests	Leave, attendance, TA, chat, hall and others are not covered.	Add a small smoke suite for each existing module (see Step 0).
Shared files that every module touches: User model, roles, navigation layout, Dashboard controller, Project and ProjectTask models	One careless edit breaks many screens.	Edit these only by adding lines, in small separate pull requests, with extra review.

Step 0: build the safety net (Week 1 to 2, before any feature)

This step adds no features. It is what makes everything after it safe, so it goes first.

#	Task	Owner	Done when
0.1	Back up live MySQL (mysqldump or cPanel backup); restore it once on staging	Backend lead	A restore actually works
0.2	Create staging: separate MySQL database, staging subdomain on cPanel, Vercel preview	Backend + frontend	Staging has a copy of live data with emails off
0.3	Point local development to staging, never to live	All developers	Live .env not on any laptop used for testing
0.4	Branch rule: no direct push to main; pull request needs 1 review	Team lead	GitHub branch protection on
0.5	GitHub Actions: run backend tests on every pull request	Backend	Red check blocks merge
0.6	Smoke tests for existing modules (list below)	QA + backend	All green on staging
0.7	Feature-flag helper: one function, reads setting 'module.sales.enabled' etc.	Backend	Flag off hides routes and menu
0.8	Scale and security fixes from the first report (rate limits, token expiry, cache off the database, load-test baseline)	Backend	Baseline numbers recorded before we add features

Smoke tests for what exists today

Area	Status
Login / logout / password reset, role redirects	Exists (AuthTest)
Employees: create, edit, photo, disable	Exists (EmployeeTest)
Leave: apply, TL approve, admin approve, balance, carry-forward, policy	Exists (LeaveRequest, LeavePolicyEngine)
Attendance + biometric sync	Exists (Biometric tests)
Projects, tasks, access rules	Exists (Project, ProjectTask, ProjectAuthorization)
WFH, Travel Allowance, Holidays, Calendar	**Missing - add**
Hubstaff page, BugSmart bug tracker, task catalog	**Missing - add**
Chat, notifications, announcements, recognitions/hall, issues	**Missing - add**
Browser check: open each main page as each role, no error (Playwright, ~20 pages)	**Missing - add**

Result: a one-command check ('did I break anything?') that takes minutes, run before every merge.

What to build first, and why

Order follows dependency: a later step cannot work without the earlier one. We also start with the parts that touch existing code the least.

#	Phase	Touches	Risk to old modules	Why here
0	Safety net + hardening	Config only	None	Everything after depends on it.
1	Accounts: payments, targets, AMC	New tables, new pages	**Very low**	Self-contained. Creates the payment status that the gate needs. Good first real release.
2	Sales: leads, project intake, inbox	New tables, new pages	**Very low**	Creates the project record that flows to Accounts. Together with step 1 gives the first working loop: Sales creates, Accounts approves.
3	PMO + Project Coordinator core: accept project, budget per department, tabbed project view, payment gate	Extends Project	**Medium**	First time we touch the existing Project model. Add-only columns, flag-controlled. Existing project pages stay as they are.
4	UI / UX / Dev workflow: pages, sections, sprints, effort approval, QA hand-off, corrections with budget	Extends tasks, corrections, bugs	**High**	Closest to what teams use daily. Built as a parallel flow; old task screens keep working until the pilot team is happy.
5	TL and resource screens: inbox, ratings, feedback, team list	Reads existing data	Medium	Reuses Hubstaff, leave and task data. Mostly read-only on old modules.
6	Dashboards, customer feedback, appreciation approval	New screens	Low	Needs real data from steps 1 to 5 first.
7	HR performance, full regression, load test, rollout	Read-only on old modules	Low	Final proof before wide release.

Phases 1 and 2 are the cheapest and safest. They give the manager something real in about 7 weeks while the risky Phase 3 and 4 work is prepared.

Order for the first release (Phases 1 and 2)

• Weeks 3-4: Accounts database tables, payment screens, payment-approved notification.

• Weeks 5-6: Sales leads, project intake, sales inbox.

• Week 7: connect them (Sales project -> Accounts approves -> notification back). Pilot with 2 sales and 1 accounts user on staging, then live with the flag on only for them.

The release routine (every phase, every time)

#	Step	Why
1	Build on a feature branch, flag OFF by default	Merging cannot change live behaviour.
2	Pull request: tests + smoke suite must pass; reviewer checks any edit to shared files	Catches breakage before merge.
3	Deploy to staging, run the old-module smoke suite and the new tests	Real-world check on a data copy.
4	QA signs off with a written checklist for the phase	A named person owns 'safe to ship'.
5	Back up live MySQL, then deploy during a quiet time (not Monday morning, not payroll or leave-cycle jobs)	Rollback point exists.
6	Run migrations; confirm they are add-only; flag still OFF	Old screens unaffected.
7	Turn flag ON for a pilot group only; watch errors for 3 to 5 days	Small blast radius.
8	Turn on for everyone	Gradual rollout complete.

If something goes wrong

• **First move: switch the flag off.** The new module disappears, old portal is as before. Takes seconds, no redeploy.

• If the code itself is faulty: revert the pull request and redeploy the previous version (on the server: git revert, pull, and run migrations rollback only if a migration was added).

• Database: because changes are add-only, the old code runs fine with the new columns present. We do not need to roll the database back. Restore from backup only if data was damaged.

• Write a short note after any incident: what happened, why the net missed it, which test now covers it.

Specific cautions per touched area

Area	Rule
Roles and permissions	Create new roles only. Never change existing role-permission rows. Test each old role after every roles migration.
User model / users table	Add optional columns only. Do not change login, status or team fields.
Project and ProjectTask	New fields nullable. Existing statuses (Planning, Active, On Hold, Completed, Cancelled) stay valid; new states are added as a separate field, not by redefining old ones.
Navigation (layout.tsx)	Add menu items inside the role+flag check. Do not reorder or rename existing items.
Dashboard controller	Add new role dashboards as separate endpoints; leave the current dashboard endpoint alone.
Notifications	New notification types only. Same delivery path, so nothing existing changes.
Scheduled jobs	Add new jobs, keep 'withoutOverlapping'. Do not touch leave, biometric or retention jobs.
Performance	Every new list is paginated and indexed; every new dashboard cached. Re-run load test at the end of each phase and compare with the Step 0 baseline.

Next 10 working days

• Day 1-2: remove or archive the old backend copy; set production mode (APP_ENV=production, APP_DEBUG=false); MySQL backup + restore test; create staging DB and backend.

• Day 2-3: branch protection, GitHub Actions, feature-flag helper.

• Day 3-7: smoke tests for missing modules (WFH, TA, Hubstaff, BugSmart, chat, hall, issues) and a Playwright page check.

• Day 7-9: hardening fixes (rate limits, token expiry, cache) and load-test baseline.

• Day 10: review with the manager; confirm answers to the open questions from the first report; start Phase 1 (Accounts).

Approve this plan, and Step 0 can start today.